张量记忆MemOS调研

> 公司官网记忆张量-推动AI应用更广泛落地
>
> 这个公司还发布了一个记忆幻觉评估框架值得关注 HaluMem

产品介绍

产品亮点

LoCoMo基准sota表现

- LoCoMo是用来评估超长上下文对话能力的数据集，平均对话长度600轮，16Ktokens，因此非常考验模型的记忆能力。

产品用法

买到他们的API key后，将原始对话存入MemOS

存储原始对话

如下方给您的示例，我们假设是在某年6月10号发生的，你只需要把用户的原始对话记录存入 MemOS，MemOS 会自动抽象、加工，并保存为记忆

python

```python
data = {
    "user_id": "memos_user_123",
    "conversation_id": "0610",
    "messages": [
      {"role": "user", "content": "我暑假定好去广州旅游，住宿的话有哪些连锁酒店可选？"},
      {"role": "assistant", "content": "您可以考虑【七天、全季、希尔顿】等等"},
      {"role": "user", "content": "我选七天"},
      {"role": "assistant", "content": "好的，有其他问题再问我。"}
    ]
  }
headers = {
  "Content-Type": "application/json",
  "Authorization": f"Token {os.environ['MEMOS_API_KEY']}"
}
url = f"{os.environ['MEMOS_BASE_URL']}/add/message"

res = requests.post(url=url, headers=headers, data=json.dumps(data))

print(f"result: {res.json()}")
```

需要时检索记忆

检索相关记忆

如下方给您的示例，我们假设是在某年9月28号发生的，用户在一个新的会话中，提出让AI推荐国庆旅游计划，MemOS 会自动召回相关记忆供AI参考，从而推荐更加个性化的旅游计划

python

```python
data = {
  "query": "我国庆想出去玩，帮我推荐个没去过的城市，以及没住过的酒店品牌",
  "user_id": "memos_user_123",
  "conversation_id": "0928"
}
headers = {
  "Content-Type": "application/json",
  "Authorization": f"Token {os.environ['MEMOS_API_KEY']}"
}
url = f"{os.environ['MEMOS_BASE_URL']}/search/memory"

res = requests.post(url=url, headers=headers, data=json.dumps(data))

print(f"result: {res.json()}")
```

产品/算法评测

主要见这个评测数据出处https://huggingface.co/datasets/MemTensor/MemOS_eval_result

LoCoMo 长对话记忆基准，评估超长对话下的记忆能力

PrefEval 偏好记忆评测，评估llm个性化记忆能力

PersonaMem 用户画像记忆评测，考查模型推理**动态用户画像**并个性化应答的能力

LongMemEval 长程记忆综合评测：**五大能力**：跨会话偏好记忆、多会话一致性、时间推理、知识更新、用户指令记忆

应用场景

多轮对话、知识演化、个性化、跨平台

1. 提升多轮对话 & 跨任务连续性表现
2. 知识演化 & 持续更新，主要强调用更新的知识替换旧知识，以及回档旧知识
3. 个性化 & 多角色建模
4. 让 MemOS 扮演跨平台记忆层，类似“统一身份下的统一记忆”，而不是每个 app 一份不互通的 history。

相关竞品

Letta 出自MemGPT作者，开源

Mem0 YC孵化器的一个初创公司

核心技术总结

> 和算法分析有重叠，主要是为了快速了解全貌

系统层面重新定义“记忆”这件事：
把各种形式的记忆（文本、激活、参数）抽象成可调度、可治理、可演化的“MemCube”，再用一个“操作系统式”的架构去统一管理和调度它们，从而支撑长程推理、知识演化、个性化和跨平台记忆迁移。

MemOS框架的三个层次

采用类似传统操作系统的分层设计。对于用户的感知来说，只有一个存记忆入口和取记忆的出口，这些层都是透明的。

从实现的角度，有三层：

Interface Layer 接口层：将模糊请求解析为标准化的**MemoryCall**内部指令格式

Operation Layer 操作层：中枢决策层，负责判断“**哪些记忆应被激活**”、“**哪些应淘汰**”、“**哪些应转存**”等，并执行具体调度

这一层具体又包括：MemReader / MemOperator / MemScheduler / MemLifecycle

Infrastructure Layer 基础设施层：提供底层存储和治理支撑，相当于“记忆数据中心”

具体包括：MemVault： 实现冷热分层存储 / *MemGovernance：* 提供系统级的权限与合规控制。/ *MemLoader*、*MemDumper：* 负责记忆的跨节点迁移与共享

算法分析

三种记忆类型以及统一管理它们的数据结构抽象：MemCube

三类原生记忆类型： MemOS 明确划分了三种基础记忆类型，并将它们纳入同一套调度框架，这三种记忆可以相互转换。

Plaintext Memory（明文记忆）： 外部注入的结构化或非结构化信息，如提示模板、知识图谱节点、用户笔记等。这类记忆可编辑性高，更新成本低，适合快速写入和修改。

Activation Memory（激活记忆）： 推理过程中产生的中间激活状态（例如 KV-cache 缓存、隐藏层向量），用于复用上下文并追踪长程依赖，相当于**高速工作记忆**。

Parameter Memory（参数记忆）： 存储于模型权重或参数增量（如 LoRA、Adapter）中的知识模块，推理效率最高、冗余最少，适合长期稳定的知识/能力表示。

系统根据调用频率、时间衰减、上下文耦合度等行为指标，对记忆进行热度建模，并据此自动触发跨类型迁移。论文称之为“跨模态演化机制（Cross-Modality Memory Transformation）”。

例如：

- 高频调用的 Plaintext 记忆片段可转化为 Activation Memory，直接以 KV-cache 形式注入模型，减少冗余前缀处理；
- 持续活跃的 Activation 可被系统后台调度转化为 Parameter Memory（如 distill 成 LoRA patch），从而释放上下文窗口；
- 冷却后的 Parameter 也可被解构回 Plaintext，重新回归易修改的表示形式。

这种闭环机制不仅提升了推理效率，更实现了知识的“热更新”与“冷淘汰”的统一治理。

MemCube 记忆单元： 通过 MemCube 抽象，所有异构记忆都被封装为结构一致的“资源单元”，供系统统一调度。仅读取元数据，调度器即可决策如**版本回滚**、**权限校验**、**冷存迁移**等操作，真正实现“**治理即调度**”。正如论文定义所述，MemCube 是“用于记忆资源的通用封装单元，具备标准化接口、行为属性和治理策略”。

模仿操作系统的inode，封装和管理上述三种记忆资源。

```
{
  "meta": { ... 一堆元数据 ... },
  "payload": { ... 实际记忆内容 ... }
}
```

Memory Payload：存真正的数据，可以是

- 文本（明文），
- 激活向量（KV / tensor），
- 参数 patch（LoRA / adapter / delta）。

Meta 只靠元信息就可以完成所有的决策和调度工作，这是最核心的部分。

从MemCube的视角观察整体的工作流程

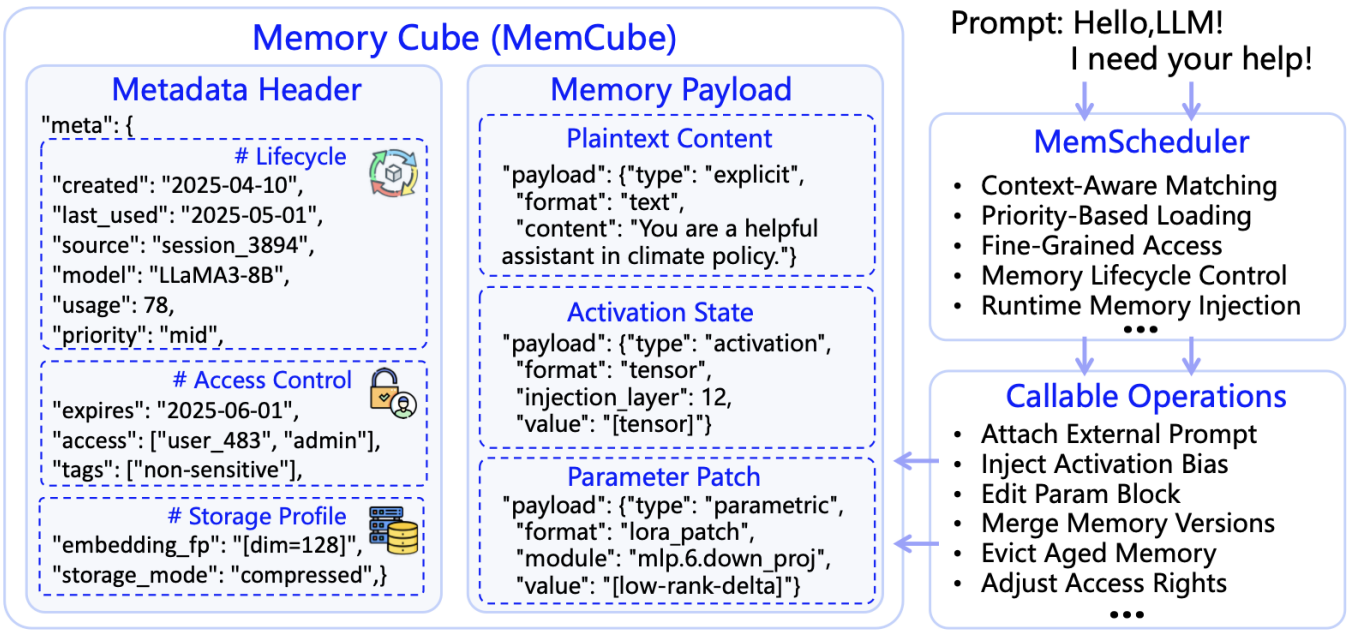

Figure 6 `MemCube`: A unified encapsulation structure for heterogeneous memory scheduling. Each `MemCube` consists of a structured Metadata Header (supporting lifecycle, permission, and storage policy) and a Memory Payload (encapsulating plaintext, activation states, or parameter deltas). It is the minimal memory unit within MEMOS that can be scheduled and composed for downstream reasoning.

Metadata Header：是这篇论文的“灵魂”，包括三大类：

1. *Descriptive Identifiers*：时间戳、来源（用户输入 / 检索 / 推理抽取 / 微调）、语义类型（事实 / 用户偏好 / 任务提示等）。
2. *Governance Attributes*：权限（谁能读写）、生命周期策略（TTL / 衰减规则）、优先级、安全 / 合规标签、水印、审计信息。
3. *Behavioral Usage Indicators*：访问频率、最近使用时间、上下文指纹、版本链等，用来驱动调度与演化。

通过这些元数据，MemOS 可以做几件“很 OS 味道”的事：

- 热点记忆上升到高带宽层（比如从明文→激活）。
- 稳定、常用的知识蒸馏成参数模块（明文/激活→参数）。
- 冷门或过时的参数外放成明文、减少结构负担（参数→明文）。
- 每一个记忆块都有完整的版本链，可以回滚、冲突解决、审计。

> 换句话说：MemCube 是“可进化的带策略的小方块”，一切记忆都被拆成这样的块来统一调度。

系统架构：MemReader / MemOperator / MemScheduler / MemLifecycle

MemReader：从交互中“提炼记忆”，提取出来的内容会被打包成 MemCube，进入记忆层

MemOperator：

MemScheduler：像进程调度一样调“记忆”

- **根据任务语义决定使用哪种记忆类型**
  - 多轮对话偏向 KV cache 回放（Activation Memory）。
  - 专家型流程偏向参数模块（Parameter）。
  - 临时/事实型问答偏向明文（Plaintext）。
- **在三种记忆之间做转换与迁移**
  - 稳定且频繁的明文记忆 → KV cache / 参数模块。
  - 过期或冲突的参数知识 → 降级成明文 + 版本化管理。

所有这些调度和转化过程都会回写到 MemCube 的元数据里，以便审计和后续优化。

MemLifecycle：

定义记忆的生命周期：生成 → 激活 → 融合 → 归档 / 过期 / 删除。

按使用频率、上下文相关性、语义重叠等自动决定：

- 哪些该保留、合并、压缩?
- 哪些该降级为冷存储、甚至删除?

系统整体执行流程见下图

5.2 Execution Path and Interaction Flow of MemOS

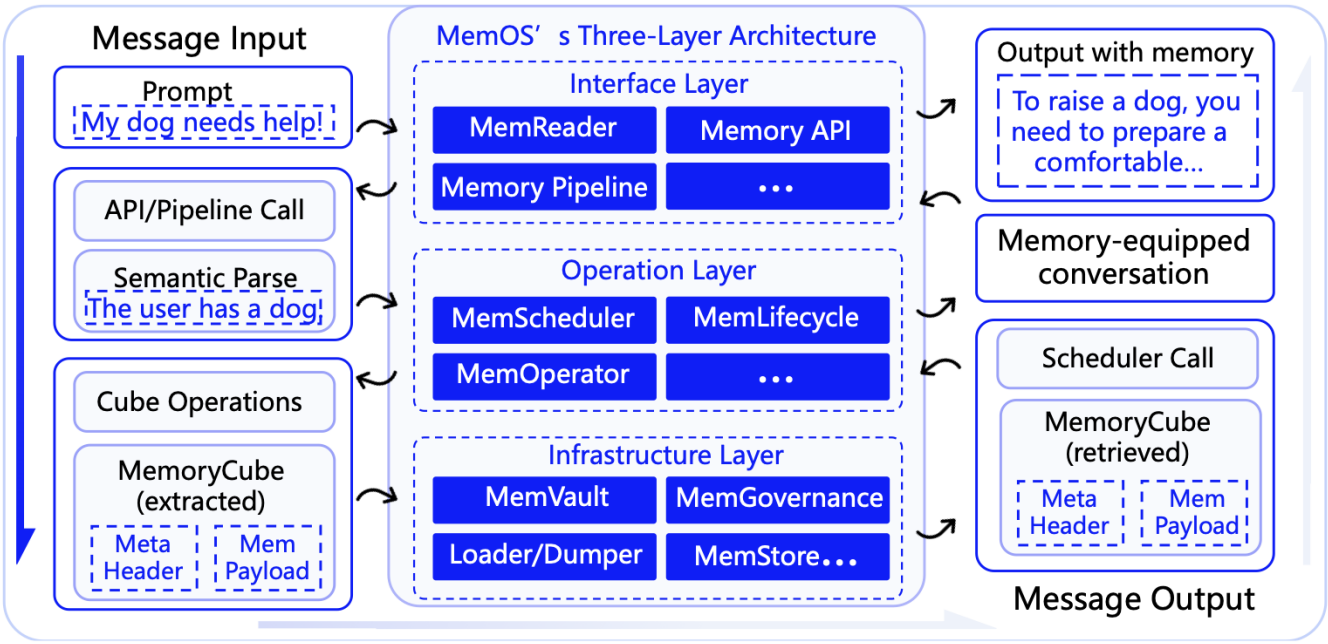

Figure 8 Overview of MemOS architecture and memory interaction flow. The system is composed of the interface layer, operation layer, and infrastructure layer. From left to right, it shows the complete memory processing pipeline from user input to parsing, scheduling, injection, and response generation. Each stage corresponds to coordinated module invocation, with MemoryCube serving as the carrier across layers for structured, governable, and traceable memory lifecycle management.

参考资料

MemOS: A Memory OS for AI System

让 LLM 拥有“可治理的记忆”：MemOS：A Memory OS for AI System 论文解读

https://huggingface.co/datasets/MemTensor/MemOS_eval_result